



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 09_2

NOMBRE COMPLETO: Del Toro Ortiz Juan Pablo Yasbeth

N° de Cuenta: 317031814

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 7

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 05/05/2026

CALIFICACIÓN: _____

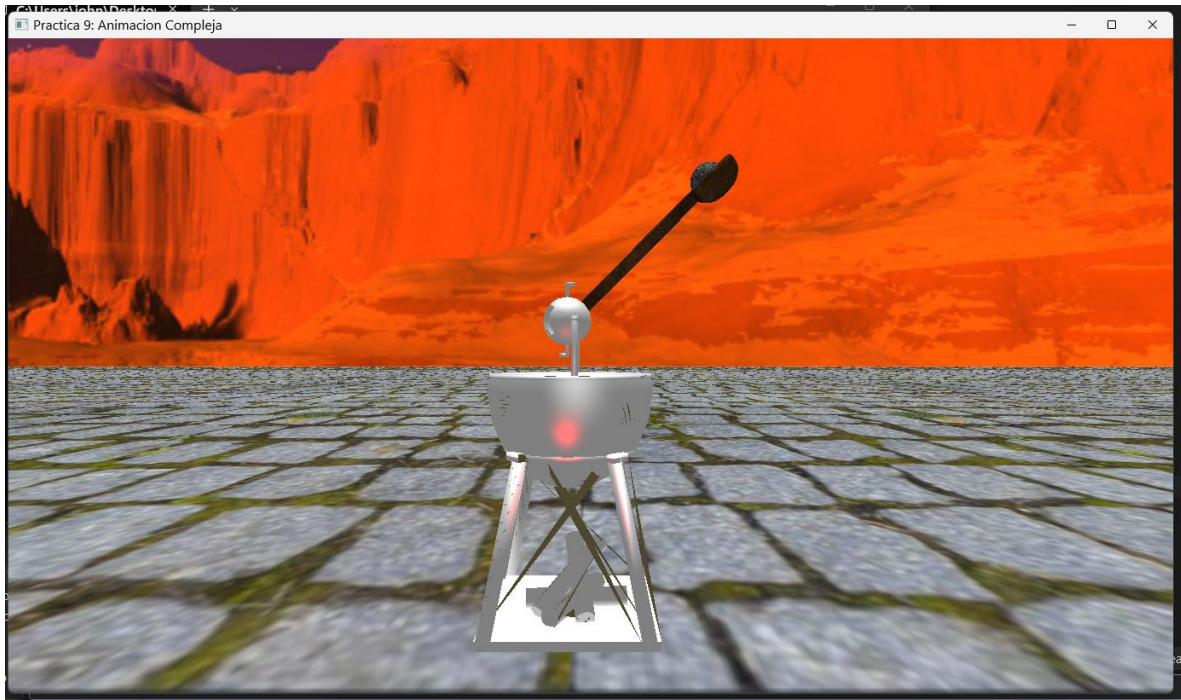
REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

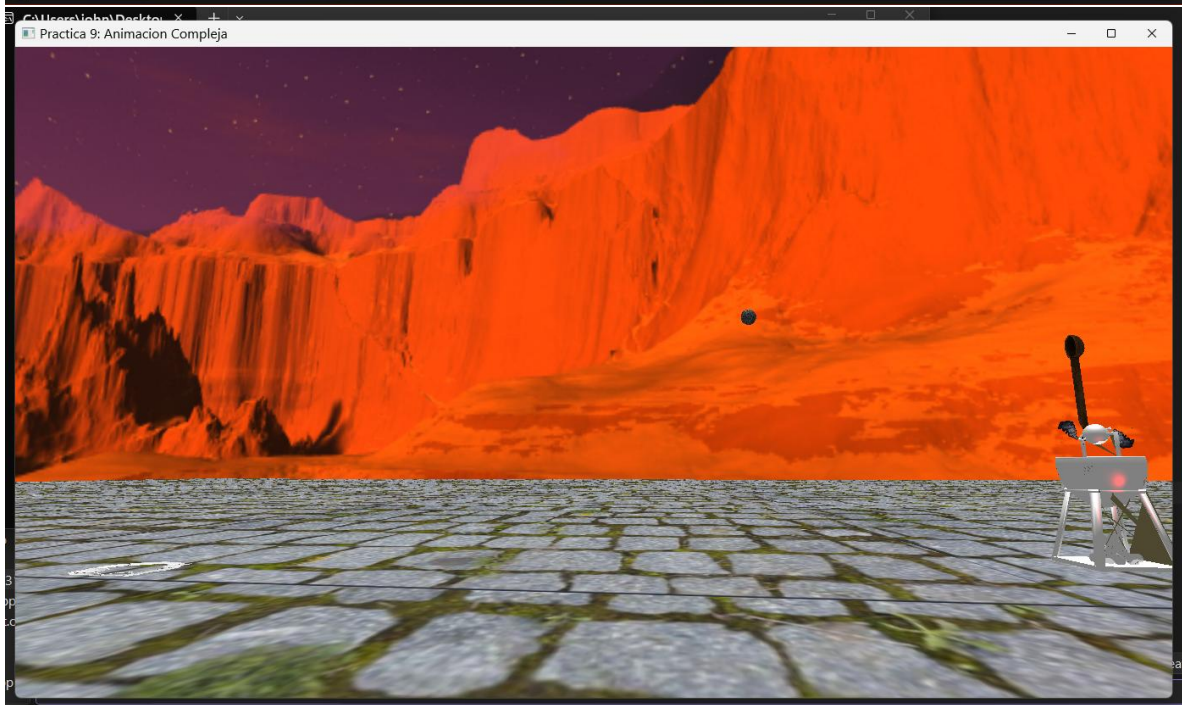
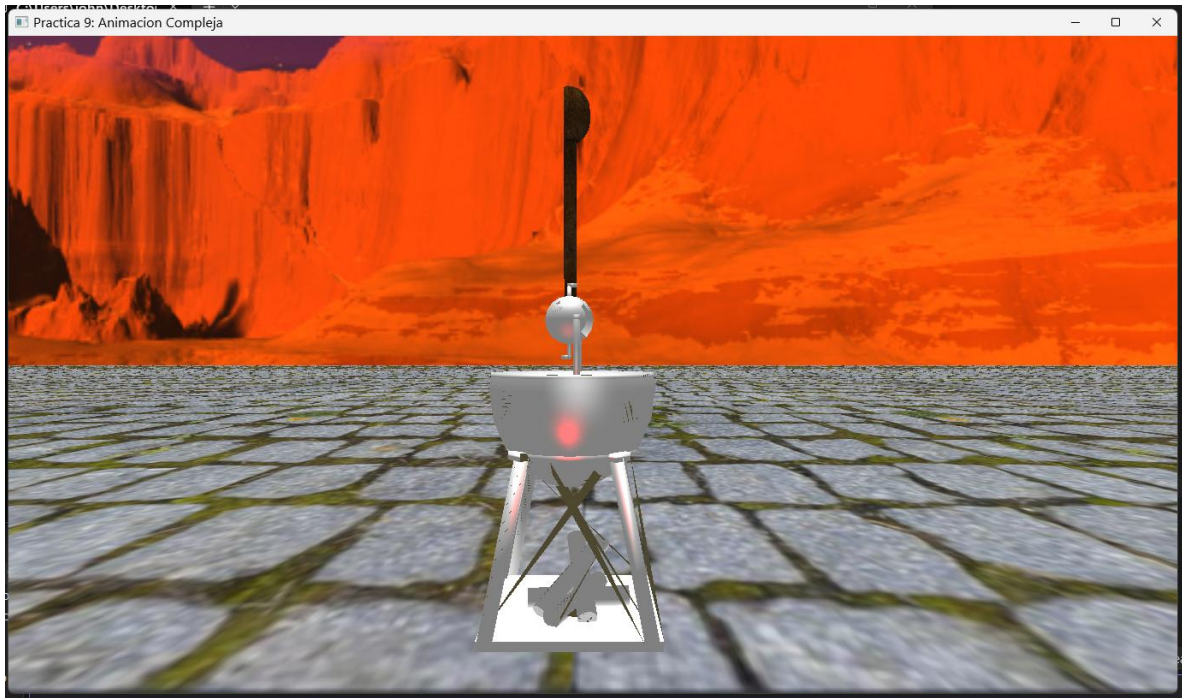
Instrucciones:

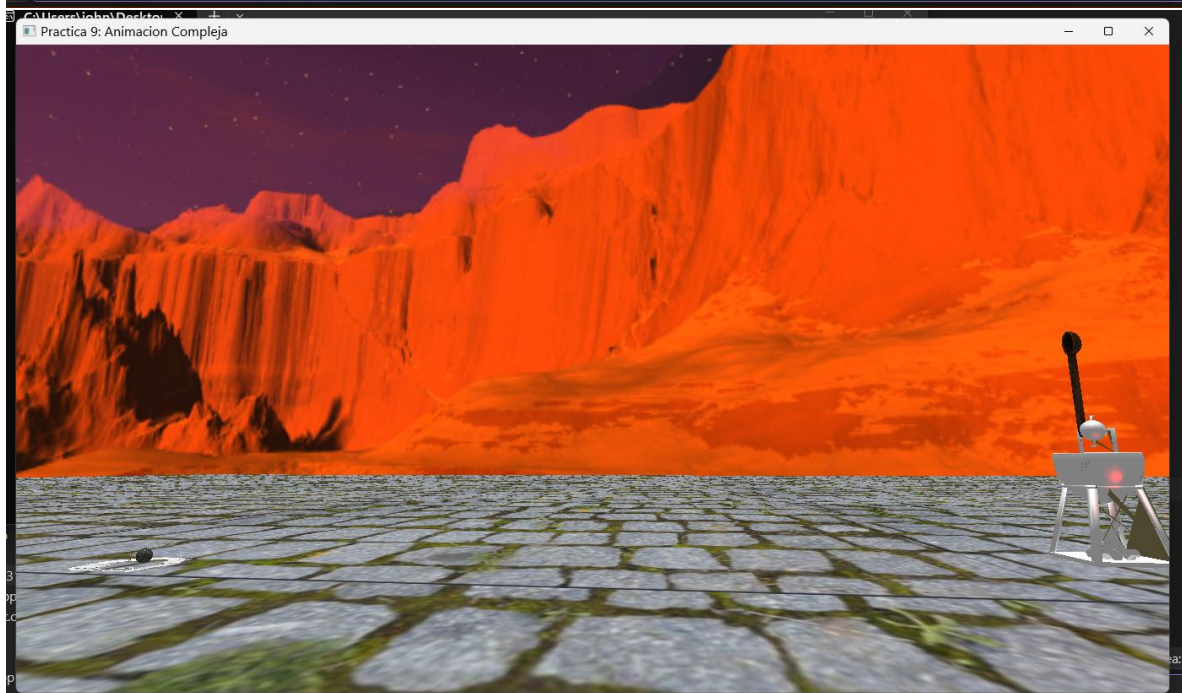
- Ejercicio 1 Humo:
 - Editar Textura para el humo
 - Blending para el humo
 - Animación del humo: no sale al inicio, se calienta el Aeolipile y empieza a salir, crece después de un tiempo y al apagarse el fuego el humo va decreciendo hasta ya no salir.
- Ejercicio 2 Brazo/palanca que lance la esfera metálica:
 - El humo crecido inicia el movimiento del brazo palanca
 - Animación del brazo para lanzar
- Ejercicio 3.- Esfera metálica:
 - Movimiento de la esfera metálica por medio de física básica: tiro parabólico y rebote
 - Al dar el último rebote la esfera entra en un "hueco" en el piso y reaparece en el brazo

Capturas de pantalla Actividad 1-3:











Video del funcionamiento:

<https://youtu.be/0K9M6mTrwGY>

Código generado:

Código de modificaciones en Window.h/Window.cpp

```
29 // Lógica de reinicio con tecla R
30 bool getReiniciarAnimacion() { return reiniciarAnimacion; }
31 void apagarReinicio() { reiniciarAnimacion = false; }
```

```
66 bool reiniciarAnimacion;
67 bool accionT;
68
```

```
44 reiniciarAnimacion = false; // Tecla R
45 accionT = false;           // Inicialización tecla T
```

```
125 // --- LÓGICA DE REINICIO (Tecla R) ---
126 if (key == GLFW_KEY_R && action == GLFW_PRESS)
127 {
128     theWindow->reiniciarAnimacion = true;
129 }
```

Código de las variables

```
54 // Variables para el fuego
55 float toffsetfuegou = 0.0f;
56 float velocidadFuego = 0.00000025f;
57 bool derechaFuego = true;
58 bool mostrarFuego = false;
59 float TIEMPO_FUEGO = 180.0f;
60
61 //Variables Para el humo
62 float toffsetHumoV = 0.0f;
63 float velocidadHumo = 0.003f;
64 bool subiendoHumo = true;
65 bool subiendoPosHumo = true;
66
67 // Variables para animación
68 float rotAeolipile = 0.0f;
69 float timerAeolipile = 0.0f;
70 bool animacionActiva = false;
71
72 // Configuración de intervalos (en segundos)
73 const float TIEMPO_ESPERA = 50.0f; // No hace nada
74 const float TIEMPO_GIRANDO = 270.0f; // Gira
75 const float TIEMPO_TOTAL = 500.0f; // Se detiene (Espera + Gira + Margen)
```

```

77 // Variables para animación de la catapulta
78 float rotCatapulta = 0.0f;
79 const float TIEMPO_CATAPULTA_INICIO = TIEMPO_ESPERA + 60.0f; // Empieza poco después que el Aeolipile
80
81 // --- Variables de la Bala ---
82 float posX_Bala = 2.5f;
83 float posY_Bala = 7.0f;
84 float posZ_Bala = -0.5f;
85 bool balaLanzada = false;
86 float rotContinuaZ = 0.0f;
87
88 // --- Variables de Estado de la Bala ---
89 int estadoBala = 0; // 0: Unida al brazo, 1: Vuelo parabólico, 2: Rebotando, 3: Detenida
90 float tiempoVueloBala = 0.0f;
91 float posX_Lanzamiento = 0.0f;
92 float posY_Lanzamiento = 0.0f;
93 float posX_Bala_Actual = 0.0f;
94 float posY_Bala_Actual = 0.0f;
95 int numRebotes = 0;
96
97 // Variables físicas
98 float velX_Bala = -4.0f; // Negativo para ir a la izquierda, mayor para más distancia
99 float velY_Bala = 6.0f; // Mayor para un arco de lanzamiento inicial más alto
100 float gravedadBala = 6.0f; // Ligeramente menor para que la bala "flote" más tiempo

```

Código Texturas y Modelos:

```

101
102 //Variables texturas
103 Texture humoTexture;
104 Texture fuegoTexture;
105 Texture AgujeroTexture;
106
107
108
109 //Modelos aelopile
110 Model Aeolipile_base_M;
111 Model Aeolipile_M;
112 //modelos de catapulta y esfera
113 Model Catapulta_M;
114 Model Bala_M;

```



```

333 //Carga texturas
334 humoTexture = Texture("Textures/humo.tga");
335 humoTexture.LoadTextureA();
336 fuegoTexture = Texture("Textures/fuego.tga");
337 fuegoTexture.LoadTextureA();
338 AgujeroTexture = Texture("Textures/agujero.tga");
339 AgujeroTexture.LoadTextureA();
340
341 //Carga modelos
342 Aeolipile_base_M = Model();
343 Aeolipile_base_M.LoadModel("Models/Aeolipile_base.obj");
344 Aeolipile_M = Model();
345 Aeolipile_M.LoadModel("Models/Aeolipile.obj");
346 Catapulta_M = Model();
347 Catapulta_M.LoadModel("Models/catapulta.obj");
348 Bala_M = Model();
349 Bala_M.LoadModel("Models/bala_cañon.obj");
350

```

Código Reinicio de animación:

```

427 // Si se presiona R, activamos la secuencia y reseteamos el cronómetro
428 if (mainWindow.getReiniciarAnimacion()) {
429     animacionActiva = true;
430     timerAeolipile = 0.0f;
431     rotAeolipile = 0.0f;
432     rotCatapulta = 0.0f;
433
434     // Reset de la bala
435     estadoBala = 0;
436     tiempoVueloBala = 0.0f;
437     numRebotes = 0;
438     velX_Bala = -8.0f; // Asegurar dirección izquierda al reiniciar
439     velY_Bala = 6.0f; // Asegurar arco alto
440
441     mainWindow.apagarReinicio(); // Apagamos la bandera para que no reinicie cada frame
442 }
443

```

Código Animación:

```
444  //--- Animación ---
445  if (animacionActiva) {
446      timerAeolipile += deltaTime;
447
448      // 1. Animación de Textura de Fuego
449      if (derechaFuego) {
450          toffsetfuegou += 0.02f * deltaTime; // Aumentado de 0.005f a 0.02f
451          if (toffsetfuegou >= 0.1f) derechaFuego = false;
452      }
453      else {
454          toffsetfuegou -= 0.02f * deltaTime; // Aumentado de 0.005f a 0.02f
455          if (toffsetfuegou <= 0.0f) derechaFuego = true;
456      }
457
458      // 2. Rotación del Aeolipile
459      if (timerAeolipile > TIEMPO_ESPERA && timerAeolipile <= (TIEMPO_ESPERA + TIEMPO_GIRANDO)) {
460          rotAeolipile += 10.0f * deltaTime;
461      }
462
463      // 3. Lógica de la Catapulta y Disparo
464      if (timerAeolipile > TIEMPO_CATAPULTA_INICIO && timerAeolipile <= (TIEMPO_ESPERA + TIEMPO_GIRANDO)) {
465          float tiempoRelativo = timerAeolipile - TIEMPO_CATAPULTA_INICIO;
466          float duracionFase = TIEMPO_GIRANDO / 4.0f;
467
468          if (tiempoRelativo < duracionFase) {
469              // Fase 1: Retroceso
470              rotCatapulta -= 0.5f * deltaTime;
471              if (rotCatapulta < -45.0f) rotCatapulta = -45.0f;
472          }
473          else {
474              // Fase 2: Lanzamiento rápido
475              rotCatapulta += 2.5f * deltaTime; // Un poco más de impulso
476
477              if (rotCatapulta >= 45.0f) {
478                  rotCatapulta = 45.0f;
479
480                  // --- DISPARO CORREGIDO ---
481                  if (estadoBala == 0) {
482                      estadoBala = 1;
483                      tiempoVueloBala = 0.0f; // REINICIO CRÍTICO: Evita que "salte" al infinito
484
485                      // Cálculo de posición inicial global (Pivote en 0.0, 4.5)
486                      float theta = rotCatapulta * toRadians;
487                      float dx = 2.5f; // Distancia horizontal de la bala al pivote en reposo
488                      float dy = 2.5f; // Distancia vertical (7.0 - 4.5)
489                  }
```

```

489
490 // Rotación de la posición relativa para obtener la global
491 posX_Lanzamiento = 0.0f + (dx * cos(theta) - dy * sin(theta));
492 posY_Lanzamiento = 4.5f + (dx * sin(theta) + dy * cos(theta));
493
494 posX_Bala_Actual = posX_Lanzamiento;
495 posY_Bala_Actual = posY_Lanzamiento;
496 }
497 }
498 }
499 }
500
501 // 4. Lógica de Físicas: Tiro Parabólico y Rebote
502 if (estadoBala == 1 || estadoBala == 2) {
503     // --- AJUSTE AQUÍ ---
504     // Cambiamos 0.12f por un valor menor (ej. 0.05f) para ralentizar el vuelo.
505     // Cuanto menor sea el número, más lenta será la parábola y el rebote.
506     tiempoVueloBala += deltaTime * 0.05f;
507

```

```

501 // 4. Lógica de Físicas: Tiro Parabólico y Rebote
502 if (estadoBala == 1 || estadoBala == 2) {
503     // --- AJUSTE AQUÍ ---
504     // Cambiamos 0.12f por un valor menor (ej. 0.05f) para ralentizar el vuelo.
505     // Cuanto menor sea el número, más lenta será la parábola y el rebote.
506     tiempoVueloBala += deltaTime * 0.05f;
507
508     // Ecuaciones de movimiento (Se mantienen igual)
509     posX_Bala_Actual = posX_Lanzamiento + (velX_Bala * tiempoVueloBala);
510     posY_Bala_Actual = posY_Lanzamiento + (velY_Bala * tiempoVueloBala) - (0.5f * gravedadBala * pow(tiempoVueloBala, 2))
511
512     // Detección de suelo
513     if (posY_Bala_Actual <= -1.5f) {
514         posY_Bala_Actual = -1.5f;
515
516         if (numRebotes < 4) {
517             estadoBala = 2;
518             numRebotes++;
519
520             // El tiempo se resetea a 0, por lo que la "lentitud" se aplica
521             // automáticamente al nuevo arco del rebote.
522             tiempoVueloBala = 0.0f;
523             posX_Lanzamiento = posX_Bala_Actual;
524             posY_Lanzamiento = posY_Bala_Actual;
525
526             velY_Bala = abs(velY_Bala) * 0.75f;
527             velX_Bala *= 0.9f;
528         }
529     }
530

```

```

534
535     if (timerAeolipile > (TIEMPO_ESPERA + TIEMPO_GIRANDO)) {
536         animacionActiva = false;
537     }
538 }
539

```


Código Instancias de los modelos:

```
594 // --- 1. AEOLIPILE BASE (Estática) ---
595 model = glm::mat4(1.0);
596 model = glm::translate(model, glm::vec3(0.0f, -0.5f, 0.1f));
597 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
598 Aeolipile_base_M.RenderModel();
599
600 // --- 2. AEOLIPILE ESFERA (Rotación en Z con R) ---
601 model = glm::translate(model, glm::vec3(0.0f, 5.0f, 0.0f));
602 model = glm::rotate(model, rotAeolipile * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
603
604 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
605 Aeolipile_M.RenderModel();
606
607 // ===== ANIMACIÓN DEL HUMO =====
608 if (subiendoHumo) {
609     toffsetHumoV += velocidadHumo;
610     if (toffsetHumoV >= 0.09f) { // Rango pequeño
611         subiendoHumo = false;
612     }
613 }
614 else {
615     toffsetHumoV -= velocidadHumo;
616     if (toffsetHumoV <= 0.0f) {
617         subiendoHumo = true;
618     }
619 }
620
621 // ===== RENDERIZADO DE LAS INSTANCIAS DE HUMO =====
622 // Solo se dibujan si la animación está activa Y estamos en el intervalo de giro
623 if (animacionActiva && (timerAeolipile > TIEMPO_ESPERA && timerAeolipile <= (TIEMPO_ESPERA + TIEMPO_GIRANDO)))
624 {
625     glEnable(GL_BLEND);
626     glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
627     humoTexture.UseTexture();
628     Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
629
630     // Animación de la textura
631     toffset = glm::vec2(0.0f, toffsetHumoV);
632     glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));
633
634     // --- INSTANCIA 1 (Derecha) ---
635     glm::mat4 modelHumo1 = model; // 'model' ya tiene la rotación de la esfera
636     modelHumo1 = glm::translate(modelHumo1, glm::vec3(0.6f, 0.8f, 0.0f));
637     modelHumo1 = glm::rotate(modelHumo1, 90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
638     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelHumo1));
639     meshList[5] -> RenderMesh();
640
641     // --- INSTANCIA 2 (Izquierda - Rotada 180 en Z) ---
642     glm::mat4 modelHumo2 = model;
643     modelHumo2 = glm::rotate(modelHumo2, 180.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
644     modelHumo2 = glm::translate(modelHumo2, glm::vec3(0.6f, 0.8f, 0.0f));
645     modelHumo2 = glm::rotate(modelHumo2, 90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
646     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelHumo2));
647     meshList[5] -> RenderMesh();
648     glDisable(GL_BLEND);
649 }
```

```

652 // ===== RENDERIZADO DEL PLANO DE FUEGO (Vinculado a R) =====
653 if (animacionActiva && (timerAeolipile <= TIEMPO_ESPERA + TIEMPO_FUEGO))
654 {
655     if (derechaFuego) {
656         toffsetfuegou += velocidadFuego;
657         if (toffsetfuegou >= 0.1f) derechaFuego = false;
658     }
659     else {
660         toffsetfuegou -= velocidadFuego;
661         if (toffsetfuegou <= 0.0f) derechaFuego = true;
662     }
663
664     toffset = glm::vec2(toffsetfuegou, 0.0f);
665
666     model = glm::mat4(1.0);
667     model = glm::translate(model, glm::vec3(0.0f, 0.7f, 0.1f));
668     model = glm::rotate(model, 90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
669     model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
670
671     glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));
672     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
673
674     glEnable(GL_BLEND);
675     glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
676     fuegoTexture.UseTexture();
677     Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
678     meshList[5] -> RenderMesh();
679     glDisable(GL_BLEND);
680 }

```

```

682 // --- RENDERIZADO DE LA CATAPULTA ---
683 model = glm::mat4(1.0);
684 model = glm::translate(model, glm::vec3(0.0f, 4.5f, -0.5f));
685 model = glm::rotate(model, -90.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
686 model = glm::rotate(model, (45.0f + rotCatapulta) * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
687 model = glm::scale(model, glm::vec3(1.5f, 1.5f, 1.5f));
688 // Envío de uniformes y renderizado
689 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
690 toffset = glm::vec2(0.0f, 0.0f);
691 glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));
692 Catapulta_M.RenderModel();
693
694 // --- RENDERIZADO DE LA BALA ---
695 model = glm::mat4(1.0);
696 if (estadoBala == 0) {
697     // ESTADO 0: Unida al brazo (Movimiento Jerárquico)
698     model = glm::translate(model, glm::vec3(0.0f, 4.5f, -0.5f)); // Traslación al pivote
699     model = glm::rotate(model, rotCatapulta * toRadians, glm::vec3(0.0f, 0.0f, 1.0f)); // Rotación del brazo
700     model = glm::translate(model, glm::vec3(posX_Bala - 0.0f, posY_Bala - 4.5f, posZ_Bala - (-0.5f))); // Posición relativa
701 }
702 else {
703     // ESTADOS 1, 2 y 3: Vuelo parabólico, rebote o detenida en el suelo (Independiente)
704     model = glm::translate(model, glm::vec3(posX_Bala_Actual, posY_Bala_Actual, posZ_Bala));
705 }
706 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
707 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
708 toffset = glm::vec2(0.0f, 0.0f);
709 glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));
710 Bala_M.RenderModel();

```

```

713 // ===== RENDERIZADO DEL HUECO =====
714 glEnable(GL_BLEND);
715 glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
716 toffset = glm::vec2(0.0f, 0.0f);
717 glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));
718 model = glm::mat4(1.0);
719 model = glm::translate(model, glm::vec3(-42.0f, -1.5f, -0.2f));
720 model = glm::scale(model, glm::vec3(5.0f, 5.0f, 5.0f));
721 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
722 AgujeroTexture.UseTexture();
723 Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
724 meshList[5]->RenderMesh();
725 glDisable(GL_BLEND);

```

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

El principal error al que me afronte fue en la actividad 3, por algún motivo al momento de intentar manejar el retorno automático de la bala de cañón esta me generaba errores, no regresaba/reiniciaba y no entendí porque ya que intente basarme en el funcionamiento del fuego animado o el vapor, pero no logre solucionarlo e incluso emplee Google Gemini IA para intentar detectar mi error y luego de múltiples intentos no pude solucionarlo

Otro problema fue relacionado con hacer rotar la esfera ya que tuve pequeñas complicaciones porque no rotaba respecto a un punto pivote sino sobre su eje, luego de analizar el problema con detenimiento ya pude solucionarlo usando las coordenadas del aelopile para la rotación

Propmts empleados para la corrección problemas:

Yo: Hola, estoy teniendo problemas con mi código necesito que este me permita un retorno automático de mi bala de cañón cumpliendo con la siguiente instrucción:

3.- Esfera metálica

- Movimiento de la esfera metálica por medio de física básica: tiro parabólico y rebote
- Al dar el último rebote la esfera entra en un "hueco" en el piso y reaparece en el brazo

Este es mi código (En esta sección le mande el código a Gemini)

P09_2-317031814

 C++



Hola, estoy teniendo problemas con mi código necesito que este me permita un retorno automático de mi bala de cañón cumpliendo con la siguiente instrucción:
3.- Esfera...

Google Gemini:



Para lograr que la esfera regrese automáticamente al brazo de la catapulta después de entrar al "hueco", necesitamos implementar un nuevo estado en tu máquina de estados de la bala. Actualmente tienes estados hasta el 3 (detenida).

Aquí tienes la lógica necesaria para cumplir con la instrucción de reaparecer en el brazo:

1. Definición de nuevos estados

Modifica tu variable `estadoBala` para incluir el hundimiento y el retorno:

- **Estado 3:** La bala entra en el "hueco" (se hunde en el eje Y).
- **Estado 4:** Reseteo de variables para que vuelva a estar unida al brazo (Estado 0).

2. Actualización de la Lógica de Físicas

Reemplaza la sección del `while` donde gestionas los estados de la bala por el siguiente código:

C++



```
// 4. Lógica de Físicas: Tiro Parabólico, Rebote y Retorno
if (estadoBala == 1 || estadoBala == 2) {
    tiempoVueloBala += deltaTime * 0.05f;

    posX_Bala_Actual = posX_Lanzamiento + (velX_Bala * tiempoVueloBala);
    posY_Bala_Actual = posY_Lanzamiento + (velY_Bala * tiempoVueloBala) - (0.5f * g * tiempoVueloBala * tiempoVueloBala);

    // Detección de suelo
    if (posY_Bala_Actual <= -1.5f) {
        posY_Bala_Actual = -1.5f;

        if (numRebotes < 4) {
            estadoBala = 2;
            numRebotes++;
            tiempoVueloBala = 0.0f;
            posX_Lanzamiento = posX_Bala_Actual;
            posY_Lanzamiento = posY_Bala_Actual;
            velY_Bala = abs(velY_Bala) * 0.50f; // Reducción de rebote
            velX_Bala *= 0.8f;
        }
        else {
            // Si ya terminó los rebotes, verificamos si está cerca del hueco
            // El hueco está en x = -42.0 aproximadamente según tu código de renderizado
            estadoBala = 3;
        }
    }
}
```

C++



```
tiempoVueloBala = 0.0f;
posX_Lanzamiento = posX_Bala_Actual;
posY_Lanzamiento = posY_Bala_Actual;
velY_Bala = abs(velY_Bala) * 0.50f; // Reducción de rebote
velX_Bala *= 0.8f;
}
else {
    // Si ya terminó los rebotes, verificamos si está cerca del hueco
    // El hueco está en x = -42.0 aproximadamente según tu código de renderizado
    estadoBala = 3;
}
}
}
```

```
// NUEVO: Lógica de hundimiento en el hueco y reaparición
if (estadoBala == 3) {
    posY_Bala_Actual -= 0.1f * deltaTime; // Se hunde en el piso

    // Una vez que se hunde lo suficiente, desaparece y vuelve al brazo
    if (posY_Bala_Actual < -5.0f) {
        estadoBala = 0; // Vuelve a estar unida al brazo
        numRebotes = 0;
        tiempoVueloBala = 0.0f;
        // Restauramos velocidades originales para el próximo tiro
        velX_Bala = -8.0f;
        velY_Bala = 6.0f;
    }
}
```

3. Ajuste en el Renderizado de la Bala

Para que la bala se vea entrando al hueco correctamente, asegúrate de que cuando esté en

`estadoBala == 3`, siga usando la posición `Actual`:

C++



```
// --- RENDERIZADO DE LA BALA ---
model = glm::mat4(1.0);
if (estadoBala == 0) {
    // UNIDA AL BRAZO: Movimiento Jerárquico
    model = glm::translate(model, glm::vec3(0.0f, 4.5f, -0.5f));
    model = glm::rotate(model, rotCatapulta * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
    model = glm::translate(model, glm::vec3(posX_Bala - 0.0f, posY_Bala - 4.5f, 0.0f));
}
else {
    // VUELO, REBOTE O HUNDIMIENTO: Independiente
    model = glm::translate(model, glm::vec3(posX_Bala_Actual, posY_Bala_Actual, 0.0f));
}
// ... resto del código de renderizado (scale, uniform, draw)
```

Resumen de cambios:

1. **Hundimiento:** En el `estadoBala == 3`, en lugar de detenerse, restamos valor a `posY_Bala_Actual` para simular que entra en el hueco.
2. **Teletransportación:** Cuando la posición Y es menor a -5.0 (fuera de la vista del usuario), cambiamos el estado a `0`.
3. **Ciclo:** Al ser `estado 0` nuevamente, el código que ya tienes para dibujar la bala unida al brazo se activará automáticamente, dejándola lista para el siguiente disparo.

¿Necesitas que ajuste también la posición X final para que coincida exactamente con el centro de la textura del agujero antes de hundirse?

Nota:

La conversación con Gemini siguió, pero no logre solucionar el problema a si que deje el reinicio básico con la Tecla R

3.- Conclusión:

- a. Los ejercicios del reporte: Complejidad, Explicación.

En esta ocasión la mayor complejidad radico en la rotación de la esfera/bala de cañón ya que esta rotaba sobre su eje en lugar de hacer el movimiento deseado y el segundo gran reto se presento con intentar automatizar el reinicio de la esfera el cual por algún motivo no funciono

- b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica

En general sentí la explicación y el tiempo para la práctica adecuado, no obstante tuve un error que por más días que intente solucionarlo no pude detectar el error que cometí así que el final decidí simplificar el funcionamiento de la animación

- c. Conclusión de la práctica:

De esta práctica puedo concluir que he aprendido como empleando expresiones matemáticas que modelan movimientos físicos se pueden desarrollar animaciones complejas las cuales en lugar de

introducir una gran cantidad de variables permiten la automatización de movimientos los cuales a su vez tienden a tener un comportamiento más natural ya que se construyen sus movimientos en base a la realidad con expresiones matemáticas y si bien estas pueden parecer un tanto complejas al momento en que se emplean permiten un manejo sencillo de comportamientos mucho más complejos.

Bibliografía

Información de tiro parabólico:

- Fernández, J. L. (n.d.). Movimiento parabólico | Fisicalab. Fisicalab. <https://www.fisicalab.com/apartado/movimiento-parabolico>

Modelos:

- Game ready medieval catapult | CGTrader. (n.d.). CGTrader. <https://www.cgtrader.com/items/974277/download-page?authuser=0>
- Free Cannonball 3D Model - TurboSquid 1726693. (n.d.). <https://www.turbosquid.com/es/3d-models/cannonball-3d-model-1726693?authuser=0>